OrderHub

Technical Guide what the app is, how it is built, and how every part works from end to end

Written so that a beginner can follow it, with enough technical depth for developers.

Repository: inbox-chronicle Stack: React TanStack Start Supabase Gmail API Gemini AI

October 2026

Contents

1. What is OrderHub? 3

2. Key ideas for beginners 4

3. Tech stack what is used for what 5

4. Architecture the big picture 7

5. End-to-end flow, step by step 8

6. Database design 11

7. Security and privacy 11

8. Reliability: limits, batching and errors 13

9. How to run the project 14

10. Current limitations and next steps 16

1. What is OrderHub?

Most of us order from many apps Amazon, Flipkart, Swiggy, Zomato, Myntra, Blinkit and more. Every app sends emails: order placed , shipped , delivered , invoices and receipts. These emails end up scattered across the inbox, so it is hard to answer simple questions like **what did I order this month?** or **which parcel is still on the way?**

OrderHub solves this. You sign in with your Google account, press **Sync Now**, and the app:

1. finds order-related emails in your Gmail (read-only it can never send or delete mail),
2. asks an AI model (Google Gemini) to read each email and pull out the order details shop, items, price, status, tracking number, dates,
3. saves those details in a database that only you can access, and
4. shows everything on a clean dashboard with filters, an order detail page and spending charts.

What you can see in the app

Page	What it shows
Dashboard (`/`)	Summary cards: active orders, delivered this month, money spent this month, open return windows; plus recent orders.
Orders (`/orders`)	All orders with filters by platform and status.
Order detail (`/orders/:id`)	One order: products, amount, payment method, tracking and a status timeline.
Analytics (`/analytics`)	Charts: spend by platform and by category.
Search (`/search`)	Search orders by product, merchant or order id (currently searches demo data).
Settings (`/settings`)	Connected Gmail account, theme (dark/light), the **Sync Now** button, logout.
Login (`/login`)	Continue with Google button.

> **Demo mode**
> If the app is started without Supabase keys, it does not crash it shows built-in **mock orders** so you can explore the interface.

2. Key ideas for beginners

A few words appear again and again in this guide. Here they are in plain language.

Term	Plain-language meaning
Frontend	The part you see in the browser: pages, buttons, charts. Here it is a **React** app.
Backend	Code that runs on a server, not in your browser. It can keep secrets (like API keys) safe. Here the backend is a **Supabase Edge Function**.
API	A way for one program to ask another program for something over the internet, e.g. give me this user s emails (Gmail API).
Database	Organised storage for data in tables (like spreadsheets). Here: **PostgreSQL**, hosted by Supabase.
OAuth	The Sign in with Google system. Google asks you for permission, then gives the app a **token** instead of your password.
Access token	A temporary key (about 1 hour) that lets the app call Gmail on your behalf.
Refresh token	A long-lived key used to get a **new** access token when the old one expires, without asking you to log in again.
JWT	JSON Web Token a signed ticket proving who you are. Supabase gives one to the browser after login; the backend checks it.
Row Level Security (RLS)	A database rule that makes each user see **only their own rows**, enforced by the database itself.
Edge Function	A small piece of backend code that Supabase runs on demand when it receives an HTTP request.
LLM / Gemini	A large language model an AI that reads text and can answer in a structured format such as JSON.
JSON	A simple text format for data, e.g. `{"merchant": "Amazon", "total_amount": 499}`.
SSR	Server-side rendering the server builds the first HTML of a page so it loads quickly.

3. Tech stack what is used for what

The project is a modern TypeScript web application. Each technology has one clear job:

3.1 Frontend (runs in the browser)

Technology	Role in OrderHub	Why it was chosen
React 19	Builds the user interface out of reusable components (order card, timeline, stat card).	Most popular UI library; huge ecosystem.
TypeScript	JavaScript with types. `src/types/order.ts` defines what an Order looks like.	Catches mistakes before the app runs.
TanStack Start	Full-stack React framework: server-side rendering, server entry (`src/server.ts`), middleware (`src/start.ts`).	Fast first load, works on Cloudflare Workers.
TanStack Router	File-based routing: each file in `src/routes/` becomes a page (e.g. `orders.$id.tsx` `/orders/:id`).	Type-safe links and URL params.
TanStack Query	Fetches and caches data: `useOrders()`, `useAuthSession()`. After a sync, the `orders` cache is invalidated so the screen refreshes.	Handles loading states, caching and refetching for us.
Tailwind CSS 4	Styling with utility classes (`rounded-xl border p-4`). Theme colours live in `src/styles.css`.	Fast to build consistent designs.
Radix UI / shadcn/ui	Accessible building blocks in `src/components/ui/` (switch, dialog, dropdown).	Keyboard and screen-reader friendly.
lucide-react	Icons (mail, refresh, moon).	Clean, consistent icon set.
Recharts	Bar and pie charts on the Analytics page.	Simple React chart components.
date-fns, zod, react-hook-form	Date helpers, data validation and forms (available from the template).	Standard utilities.

3.2 Backend and services

Technology	Role in OrderHub
Supabase	Backend as a service : provides login (Auth), the Postgres database, auto-generated REST API and Edge Functions all in one project.
Supabase Auth + Google provider	Handles Sign in with Google , stores the user s session and gives the browser a JWT.
PostgreSQL	Stores orders, products, timeline events and sync state (`supabase/schema.sql`).
Row Level Security	Policies like `auth.uid() = user_id` make sure a user can only read their own orders.
Edge Function `sync-gmail` (Deno)	The heart of the sync: talks to Gmail and Gemini and writes to the database (`supabase/functions/sync-gmail/index.ts`).
Gmail API	Used with read-only scope `gmail.readonly` to search for and download order emails.
Google OAuth token endpoint	Swaps the stored refresh token for a fresh Gmail access token.

Technology	Role in OrderHub
Google Gemini API	AI model (default `gemini-3.8-flash`, configurable via the `GEMINI_MODEL` secret) that turns email text into order JSON.

3.3 Tooling, build and hosting

Technology	Role
Vite 8	Dev server with instant reload (`npm run dev`) and production bundler (`npm run build`).
@lovable.dev/vite-tanstack-config	Pre-made Vite setup: React, Tailwind, TanStack Start, path alias `@/`, env injection and Nitro.
Nitro Cloudflare Workers	Packages the server part of the app so it can run on Cloudflare s global edge network.
Lovable	AI/visual app editor connected to this Git repository; commits to `main` sync back to Lovable.
Supabase CLI (`npx supabase`)	Deploys the edge function and manages secrets.
ESLint + Prettier	Code quality and formatting (`npm run lint`, `npm run format`).

4. Architecture the big picture

The diagram shows every moving part and the order in which they talk to each other during a normal session.

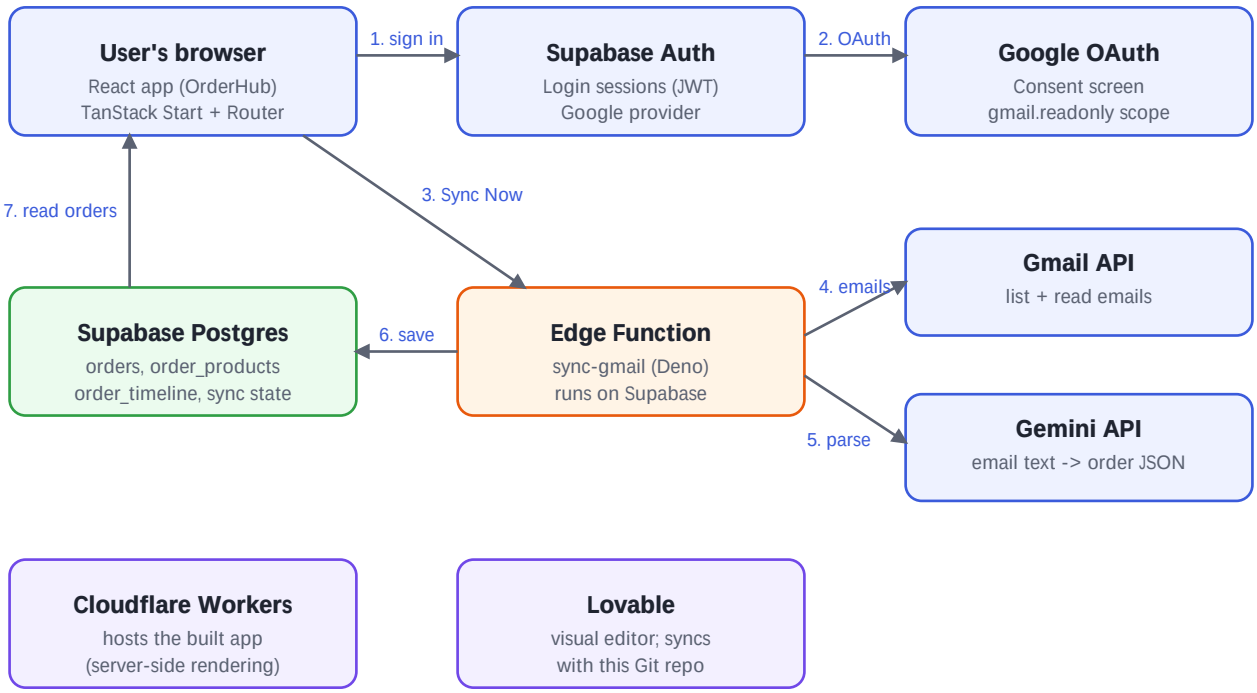


Figure 1 OrderHub architecture. Numbers show the order of a typical session: sign in, sync, read orders.

Why is the sync done in an Edge Function and not in the browser?

- **Secrets stay secret.** The Gemini API key, the Google client secret and the database service key must never be sent to a browser. The edge function keeps them on the server.
- **Trusted writes.** The function checks the user s identity (JWT) and then writes orders with the correct `user_id`, so nobody can insert orders for someone else.
- **Long-lived Gmail access.** Only the server can use the refresh token + client secret to get new Gmail tokens.

Where the code lives

```
src/
  routes/          Pages: index, orders, orders.$id, analytics, search, settings, login, __root (app shell)
  components/      App components + components/ui (shadcn/ui building blocks)
  hooks/           use-auth.ts, use-orders.ts, use-theme.ts, use-mobile.tsx
  services/        auth.ts (Google sign-in), orders.ts (database queries + data mapping)
  lib/             supabase.ts (client + token capture), mock-data.ts, format.ts, error pages
  types/order.ts   Order, Product, TimelineEvent types
  server.ts        SSR entry with error handling;  start.ts: request middleware
supabase/
  schema.sql                     Tables + Row Level Security policies
  functions/sync-gmail/index.ts  Gmail -> Gemini -> database sync
```

5. End-to-end flow, step by step

This chapter follows one user from opening the app to seeing their orders. Each step has a **simple** explanation and a **technical** one.

Step 1 Signing in with Google

Simply: you click Continue with Google , Google asks Allow OrderHub to read your email? , you accept, and you are back in the app, logged in.

Technically:

1. `src/routes/login.tsx` calls `signInWithGoogle()` in `src/services/auth.ts`.
2. That calls `supabase.auth.signInWithOAuth({ provider: "google" })` with `scopes: gmail.readonly`, `access_type: offline` and `prompt: consent`. Offline + consent makes Google return a **refresh token**.
3. The browser is redirected to Google s consent screen, then to Supabase s callback URL, then back to the app.
4. Supabase creates a session: a **JWT** for the app plus Google s `provider_token` (access token) and `provider_refresh_token`.

Step 2 Saving the Google tokens

Simply: the app remembers the key that lets it read your Gmail later, even after an hour or a page reload.

Technically: `src/lib/supabase.ts` listens to `onAuthStateChange`. Google tokens are only visible once, right after login, so the code:

- keeps the short-lived `provider_token` in memory as a fast path, and
- saves the `provider_refresh_token` into the `gmail_sync_state` table (`google_refresh_token` column). Row Level Security lets each user write only their own row.

> **Why both tokens?**
> Google access tokens expire after about an hour, and Supabase drops the Google token when it refreshes its own session. The refresh token lets the server get a new Gmail access token at any time so Sync keeps working without logging in again.

Step 3 Pressing Sync Now

Simply: the browser asks the backend: please check my Gmail for new orders . It does this in small batches and shows progress.

Technically (`handleSync` in `src/routes/settings.tsx`):

1. Gets the current Supabase session (`supabase.auth.getSession()`) to obtain the user s JWT.
2. Sends `POST {SUPABASE_URL}/functions/v1/sync-gmail` with headers `Authorization: Bearer <JWT>` and `apikey`, and body `{ provider_token, page_token, page_size: 4 }`.
3. Reads the reply `{ processed, skipped, deferred, orders, errors, nextPageToken }`, adds up the totals and invalidates the `orders` query so the dashboard refreshes.

4. Repeats with the `nextPageToken` until there are no more pages or 25 calls were made (about 100 emails per click).
5. Shows a final message, e.g. Sync complete! Processed 100 emails, found 12 orders. , including failed or postponed counts.

Step 4 Inside the sync-gmail edge function

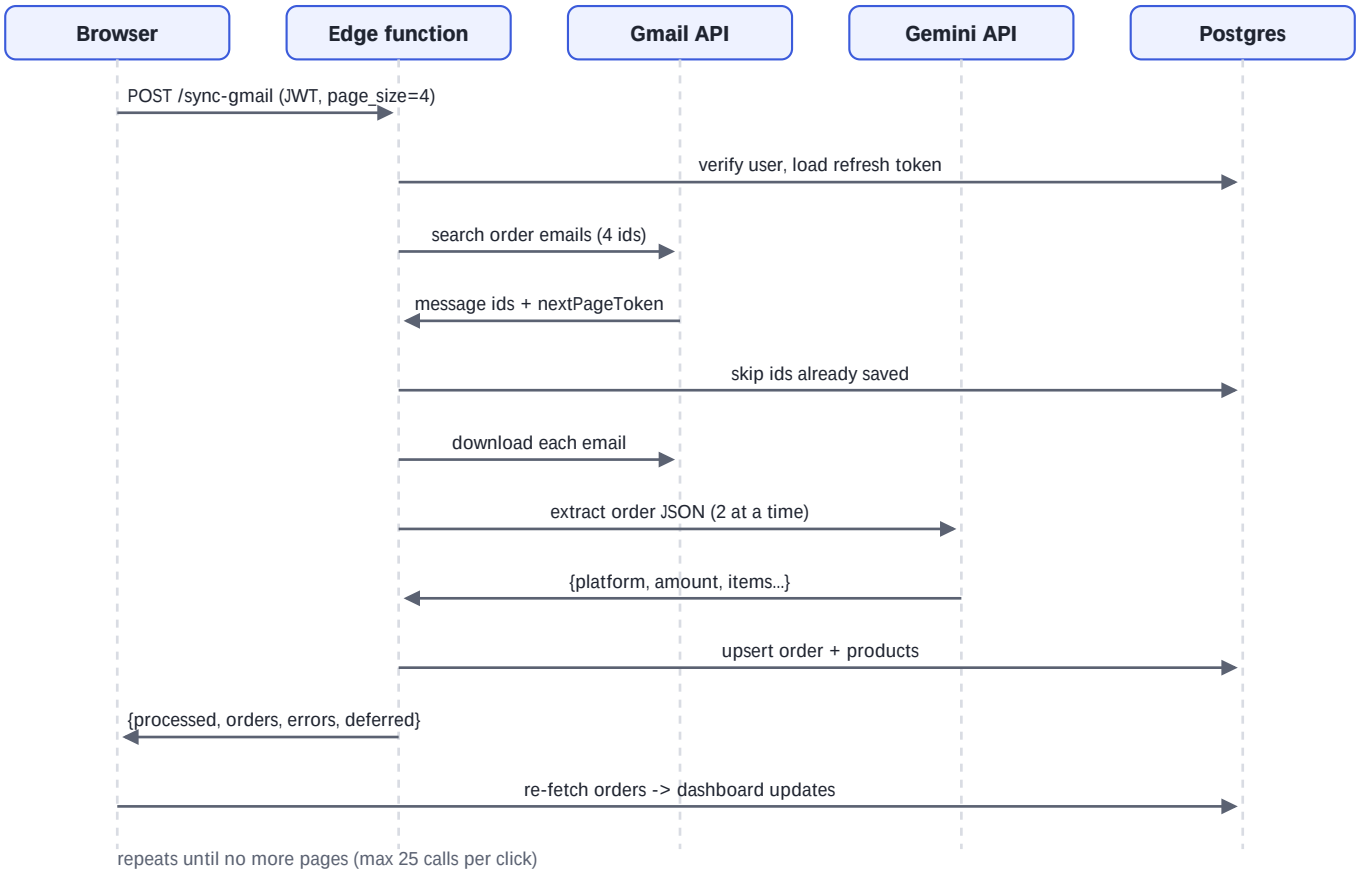


Figure 2 What happens during one sync call (one batch of up to 4 emails).

The function in `supabase/functions/sync-gmail/index.ts` runs these stages for every batch:

Stage	Simple explanation	Technical detail
a. Who are you?	Check the user really is logged in.	Creates a Supabase client with the caller s JWT and calls `auth.getUser()`. A second **admin** client (service-role key) is used for database writes.
b. Get Gmail access	Find a working key for Gmail.	Uses `provider_token` from the browser if present; otherwise (or if Gmail answers 401) it calls `https://oauth2.googleapis.com/token` with the stored refresh token + `GOOGLE_CLIENT_ID`/`GOOGLE_CLIENT_SECRET`.
c. Find order emails	Search the inbox for likely order emails.	`GET gmail/v1/users/me/messages?q=subject:(order OR receipt OR invoice OR purchase OR shipped OR delivery)&maxResults=4&pageToken=`
d. Skip known emails	Don t pay the AI twice for the same email.	Looks up `email_message_id` values already in `orders` for this user and skips them.

Stage	Simple explanation	Technical detail
e. Read each email	Download the email and turn it into plain text.	`messages/{id}?format=full`; walks nested MIME parts, prefers `text/plain`, otherwise decodes `text/html` (capped in size) and strips tags/styles.
f. Ask the AI	Gemini reads the email and fills in an order form.	`POST /models/{GEMINI_MODEL}:generateContent` with a prompt describing the JSON shape, `responseMimeType: application/json`, low thinking level, 25 s timeout, one retry on 429/500/503. Non-order emails return `{"skip": true}`.
g. Clean the answer	Fix missing or odd values.	Platform/status/category forced into allowed lists, amounts parsed to numbers, dates validated, merchant falls back to platform or sender name.
h. Save	Store the order and its items.	`upsert` into `orders` on `email_message_id` (no duplicates on re-sync), then replace rows in `order_products`.
i. Report back	Tell the browser what happened.	Updates `gmail_sync_state.last_synced_at` and returns counts, errors and `nextPageToken`.

Example: what Gemini returns for one email

```
{
  "platform": "Amazon",
  "merchant": "Amazon.in",
  "order_id": "405-1234567-7654321",
  "status": "shipped",
  "total_amount": 1299,
  "currency": "INR",
  "category": "shopping",
  "products": [{ "name": "Boat Rockerz 450", "quantity": 1, "price": 1299 }],
  "tracking_number": "AMZ12345678",
  "order_date": "2026-10-18T10:21:00Z",
  "expected_delivery": "2026-10-21T00:00:00Z",
  "payment_method": "UPI"
}
```

Step 5 Showing the orders

Simply: the dashboard asks the database for my orders and draws them.

Technically: `useOrders()` (`src/hooks/use-orders.ts`) runs `fetchOrders()` (`src/services/orders.ts`), which queries:

```
supabase.from("orders")
  .select("*, products:order_products(*), timeline:order_timeline(*)")
  .order("order_date", { ascending: false })
```

Because of Row Level Security, the database itself only returns rows where `user_id` equals the logged-in user. The rows come back in database style (`total_amount`, `order_date`) and `mapOrderRow()` converts them into the app s `Order` type (`totalAmount`, `orderDate`) used by every component.

6. Database design

All data lives in four tables in Supabase Postgres (`supabase/schema.sql`). Arrows point from a row to the row it belongs to.

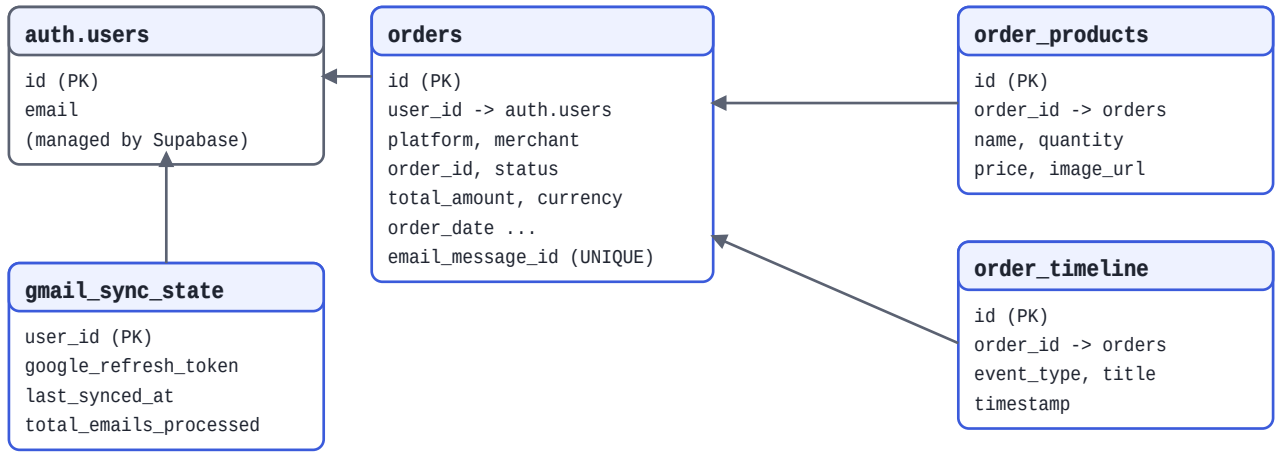


Figure 3 Tables and relationships.

Table	Purpose	Important columns
`orders`	One row per order email that was recognised.	`user_id`, `platform`, `merchant`, `status`, `total_amount`, `order_date`, `email_message_id` (UNIQUE, prevents duplicates).
`order_products`	Items inside an order.	`order_id` (links to orders, deleted with it), `name`, `quantity`, `price`.
`order_timeline`	Status history of an order (placed, shipped, delivered).	`event_type`, `title`, `timestamp`. Not filled by the sync yet.
`gmail_sync_state`	Per-user sync bookkeeping.	`google_refresh_token`, `last_synced_at`, `total_emails_processed`.

Row Level Security policies

```
CREATE POLICY "Users see own orders"   ON orders         FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users see own products" ON order_products FOR ALL
  USING (order_id IN (SELECT id FROM orders WHERE user_id = auth.uid()));
CREATE POLICY "Users see own sync state" ON gmail_sync_state FOR ALL USING (auth.uid() = user_id);
```

`auth.uid()` is the id inside the user s JWT. Even if someone changed the frontend code, the database would still refuse to return another person s orders.

7. Security and privacy

- **Read-only Gmail.** The only Gmail permission requested is `gmail.readonly`. The app cannot send, delete or modify email.
- **No passwords.** Google OAuth means the app never sees your Google password.

- **Secrets on the server only.** `GEMINI_API_KEY`, `GOOGLE_CLIENT_SECRET` and `SUPABASE_SERVICE_ROLE_KEY` exist only as Supabase secrets inside the edge function. The browser only has the public **anon key**, which is safe to expose because RLS protects the data.
- **Identity checked on every sync.** The edge function verifies the JWT before doing anything, and always writes the verified `user_id`.
- **Data isolation.** Row Level Security on every table.
- **Limited email storage.** Only extracted order fields and a 500-character snippet (`raw_email_snippet`, for debugging) are stored not whole emails.
- **Revoking access.** Remove OrderHub at myaccount.google.com/permissions at any time; the stored refresh token then stops working.

8. Reliability: limits, batching and errors

Edge functions are small and have strict limits. The sync was designed around them:

Limit / problem	How OrderHub handles it
~2 s of CPU and 250 MB memory per request (`WORKER_RESOURCE_LIMIT`)	Only **4 emails per request**, processed **2 at a time**; only the plain-text part is decoded when available; HTML is capped before cleaning.
150 s maximum request time (`Request idle timeout`)	Each Gemini call has a 25 s timeout and at most one retry; no new email is started after 60 s.
Gemini overloaded (high demand , 429/503)	Retried once; if still busy, the email is **postponed** (not saved) and picked up on the next sync.
Gemini model retired	Model name is a secret (`GEMINI_MODEL`); a model error stops the batch immediately with one clear message.
Google token expired	Automatic refresh using the stored refresh token.
Gmail permission missing	Clear message telling the user to re-authorize and tick the Gmail checkbox.
Same email synced twice	Skipped before calling Gemini; `upsert` on `email_message_id` as a safety net.
AI returns incomplete data	Values are normalised and given safe defaults so required columns are never empty.

What the user sees

Message in Settings	Meaning
Sync complete! Processed N emails, found M orders.	Success.
K email(s) postponed (Gemini busy)	Click Sync again later.
K email(s) failed to parse (e.g.)	Those emails could not be read; the example error explains why.
Gmail read permission was not granted	Re-authorize Google with the Gmail checkbox ticked.
Google access expired and no refresh token is stored	Sign out and sign in once.

9. How to run the project

9.1 What you need

- **Node.js 20+** and npm (Bun also works there is a `bun.lock`).
- A **Supabase** project (free tier is fine).
- A **Google Cloud** project with an OAuth client of type Web application .
- A **Gemini API key** from Google AI Studio.

9.2 Run the frontend

```
git clone <repository-url>
cd inbox-chronicle
npm install
```

Create a file named `.env` in the project root:

```
VITE_SUPABASE_URL=https://<your-project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<your-anon-public-key>
```

Both values are in Supabase Project Settings API. Then start the app:

```
npm run dev
```

Open the address that Vite prints (for example `http://localhost:8080`). Without the `.env` values the app runs in demo mode with mock data.

9.3 Set up the database

In Supabase **SQL Editor**, paste and run the whole of `supabase/schema.sql`. It creates the four tables, the RLS policies and the `google_refresh_token` column.

9.4 Set up Google sign-in

1. Google Cloud Console **APIs & Services** **Library** enable **Gmail API**.
2. **Google Auth Platform** **Data Access**: add scope `https://www.googleapis.com/auth/gmail.readonly`.
3. **Audience**: while the app is in Testing , add your Gmail address as a test user.
4. **Clients** your Web client add the redirect URI `https://<project-ref>.supabase.co/auth/v1/callback`.
5. Supabase **Authentication** **Providers** **Google**: enable it and paste the Client ID and Client Secret.
6. Supabase **Authentication** **URL Configuration**: add `http://localhost:8080` (and your deployed URL) to the redirect URLs.

9.5 Deploy the sync function

```
npx supabase login
npx supabase link --project-ref <your-project-ref>
npx supabase secrets set GEMINI_API_KEY=<key> GOOGLE_CLIENT_ID=<id> GOOGLE_CLIENT_SECRET=<secret>
npx supabase secrets set GEMINI_MODEL=gemini-3.8-flash      # optional
npx supabase functions deploy sync-gmail
```

9.6 Try it

1. Open the app **Continue with Google** tick Read your email messages Continue.
2. Go to **Settings** **Sync Now** and wait for the progress message.
3. Open the **Dashboard** or **Orders** page to see the extracted orders.

9.7 Useful commands

Command	What it does
`npm run dev`	Development server with hot reload.
`npm run build`	Production build (Nitro, Cloudflare Workers target).
`npm run preview`	Serve the production build locally.
`npm run lint` / `npm run format`	Check code quality / auto-format.
`npx supabase functions deploy sync-gmail`	Deploy backend changes.
`npx supabase secrets list`	See which secrets are set (values are hidden).

Supabase free tier
Free projects are **paused** after a period of inactivity. If the app suddenly cannot reach Supabase (the `*.supabase.co` address does not resolve), open the Supabase dashboard and click **Restore project**.

Lovable
This repository is connected to Lovable. Commits pushed to `main` appear in the Lovable editor, so keep `main` working and never force-push or rewrite pushed history.

10. Current limitations and next steps

Limitation	Suggested improvement
Each click scans the newest ~100 matching emails; older history isn t reached.	Save the Gmail page position in `gmail_sync_state`, then use Gmail s History API for incremental syncs.
Search uses only subject keywords.	Add known sender domains (`from:amazon.in OR flipkart.com`), more keywords (booking, ticket, trip, refund) and exclude Promotions.
Placed , Shipped , Delivered emails become separate orders.	Merge by platform + order id: update status and add `order_timeline` events.
Gemini free tier can be overloaded or rate-limited.	Use a paid key or a lighter model; postponed emails are retried automatically on the next sync.
Automatic sync toggle is cosmetic.	Schedule the function (Supabase cron) per user using the stored refresh token.
Search page uses demo data.	Switch it to `useOrders()` like the other pages.
Pages don t redirect to login when signed out.	Add an auth guard in the root route (`beforeLoad`).

Summary

OrderHub is a React + TanStack Start web app backed by Supabase. Google OAuth gives it read-only Gmail access; a Supabase Edge Function searches the inbox, asks Gemini to turn each order email into structured JSON, and saves it to Postgres, where Row Level Security keeps every user s data private. The browser then reads those orders through TanStack Query and shows them as a dashboard, list, detail pages and charts.